

Reprendre le code d'une entreprise, le **réparer**, le **tester**, le **déployer** et le **surveiller**.

Vous êtes la nouvelle équipe technique de **MutuAlp Assurances**. On vous confie **SinistreFlow**, l'application de déclaration de sinistres : 5 ans de production, une vraie base de données, des partenaires qui en dépendent... et une pile de bugs. Votre mission : remettre le produit d'aplomb et lui donner une chaîne DevOps digne de ce nom.

🕒 4 jours · 28 h

👥 Équipes de 3 à 4

🐛 22 tickets

🔧 Unitaires · Intégration · E2E

⚙️ GitHub Actions

☁️ VM Linux

📊 Prometheus · Grafana

Apprenez la tech. Construisez votre avenir.



01 Mise en situation

Lundi, 9 h. Vous arrivez chez MutuAlp Assurances, mutuelle régionale de 45 000 sociétaires. Le lead dev, Thomas, est parti il y a un mois. Il a laissé une note, un dépôt Git, un dump de la base de production... et aucune documentation d'exploitation.

425

dossiers sinistres en base

3

versions d'API en production

0

test automatisé fiable

0

métrique, alerte ou pipeline

Les assurés

Déclarent leurs sinistres (accident, vol, dégât des eaux...) via un formulaire en ligne en 4 étapes.

Les gestionnaires

Instruisent les dossiers dans le back-office : recherche, changement de statut, suivi de l'indemnisation.

Les partenaires

Le cabinet **ExpertAuto** récupère les dossiers à expertiser et dépose ses rapports via l'API. Leur connecteur tourne sur votre serveur... et **vous n'avez pas le droit de le modifier**.



La situation est critique. Le connecteur ExpertAuto est en échec : plus aucune expertise n'arrive, donc plus aucune indemnisation n'est versée. Le support a ouvert 22 tickets. Le CTO propose de « supprimer les vieilles versions de l'API pour simplifier ». La direction veut savoir *avant vendredi* si l'application peut être redéployée

proprement sur un nouveau serveur, et surtout **être prévenue la prochaine fois que ça casse.**

Ce que vous devez livrer vendredi

Code & qualité

- Les 22 tickets corrigés, **un commit par ticket au message imposé**
- Une ADR (décision d'architecture) sur le versioning de l'API
- Les 3 couches de tests : unitaires, intégration, End-to-End

Chaîne DevOps

- Un dépôt GitHub avec un pipeline **GitHub Actions** qui exécute tous les tests
- L'application conteneurisée et **déployée sur une VM**
- Le connecteur ExpertAuto **au vert, lancé depuis la VM**
- Une **supervision** : métriques, tableau de bord, alertes

02 Règles du jeu

✅ Obligatoire

- Un dépôt GitHub par équipe (formateur invité en *Admin*).
- `main` protégée : on passe **uniquement** par des Pull Requests relues par un coéquipier.
- Une branche par ticket (`fix/SF-104-reference-sequence`).
- **Un commit par correction**, avec le **message exact** donné dans le ticket. Le test qui prouve la correction va dans le même commit.
- Un **journal de bord** d'équipe (`docs/JOURNAL.md`) : qui a fait quoi, chaque jour, ce qui a bloqué.

❌ Interdit

- Modifier `partner-client/` : c'est le code d'une autre entreprise.
- Supprimer ou désactiver un test pour « faire passer » la CI.
- Modifier le dump `db/dump/*.sql` : c'est la vérité de la production.
- `git push --force` sur `main`.
- Committer un secret (mot de passe, clé, token).

Le format des commits

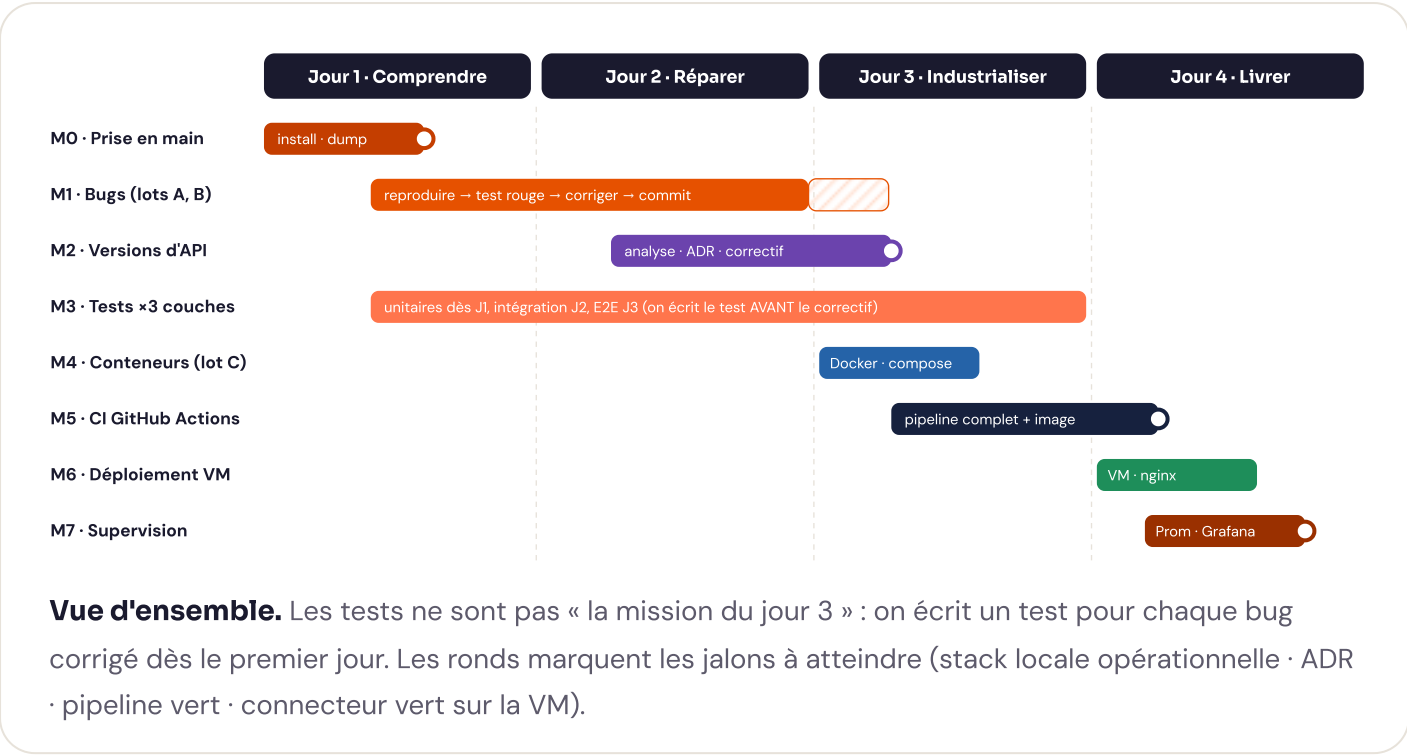
Le formateur vérifie automatiquement l'historique de chaque dépôt : un message différent d'un seul caractère n'est pas comptabilisé. Format Conventional Commits :

```
fix(SF-107): corriger le calcul de l'offset de pagination
|           |
|           | └ description EXACTE du ticket
|           └── numéro du ticket
└── type : fix · feat · test · ci · docs
```

i Pourquoi tant de rigueur ? En entreprise, l'historique Git est un outil d'exploitation : quand la production casse à 3 h du matin, on cherche *quel commit* a introduit le

problème (`git log --grep SF-` , `git bisect`) et on le relie au ticket. Un commit = une intention = un retour arrière possible.

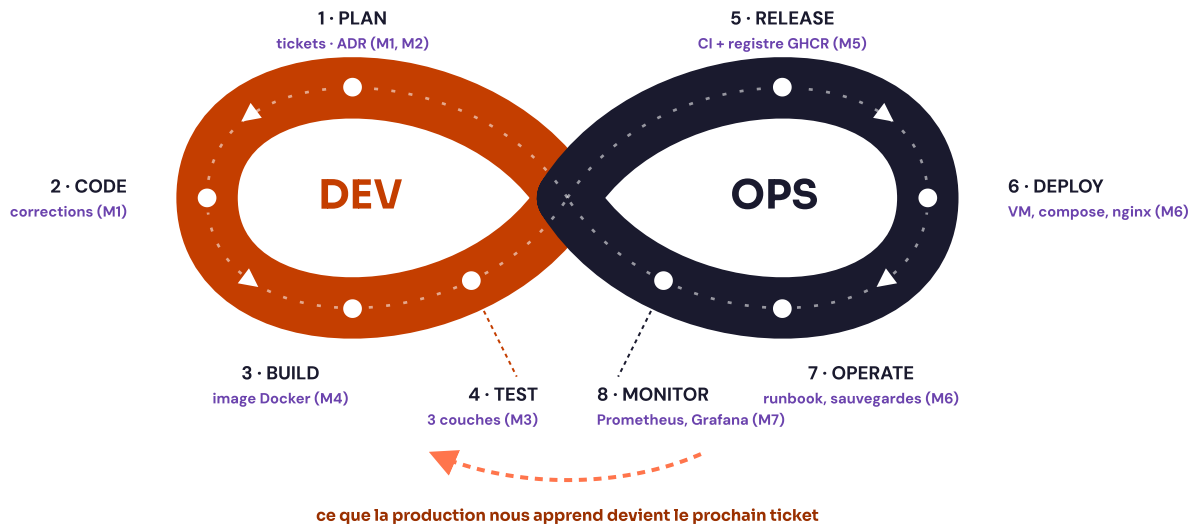
03 Planning des 4 jours



Jour	Matin	Après-midi	Point de contrôle (17 h)
J1	Mission 0 : dépôt GitHub, protection de <code>main</code> , application lancée avec le dump, connecteur lancé (rouge).	Mission 1 : premiers tickets (SF-101, 105, 107, 112, 114...) + leurs tests unitaires / d'intégration.	Stack locale + 1 ^{re} PR fusionnée.
J2	Mission 1 suite (lot A complet, lot B).	Mission 2 : SF-109, analyse, ADR, correctif. Tests d'intégration des 3 versions.	ADR fusionnée sur <code>main</code> .
J3	Mission 4 : Docker, compose, lot C. Tests E2E Playwright.	Mission 5 : GitHub Actions (unitaires, intégration, E2E, recette partenaire, image).	Pipeline vert sur <code>main</code> .
J4	Mission 6 : provisionnement de la VM, déploiement, connecteur au vert sur la VM .	Mission 7 : métriques, Grafana, alertes, test de panne, preuves dans le dépôt.	Lien du dépôt GitHub rendu.

04 Le DevOps en un schéma

Le DevOps n'est ni un outil ni un poste : c'est une façon de travailler où **ceux qui construisent le logiciel sont aussi responsables de son fonctionnement en production**, et où tout ce qui peut être automatisé l'est, pour livrer souvent, sans peur.



La boucle infinie. À gauche, la partie « Dev » ; à droite, la partie « Ops ». Le DevOps consiste à faire tourner cette boucle *vite* et *souvent*. Chaque mission du TP correspond à une étape de la boucle.

Les 5 idées à retenir

Boucles de rétroaction courtes

Plus un bug est détecté tôt, moins il coûte. Un test unitaire répond en millisecondes, la CI en minutes, un client en colère en semaines.

Shift-left

Ramener les contrôles le plus à gauche possible de la boucle : tests, sécurité, migrations, validés *avant* la production.



Automatiser la répétition

Ce qui dépend de « penser à... » finit par être oublié (SF-206). Ce qui est dans un script ou un pipeline est fait à chaque fois.



Mesurer

Sans métriques, on ne sait pas si ça marche, ni depuis quand ça ne marche plus (SF-115). On supervise la technique et le métier.



Tout est code

Infrastructure, pipeline, configuration, tableaux de bord, alertes : versionnés dans Git, relus en PR, reproductibles.



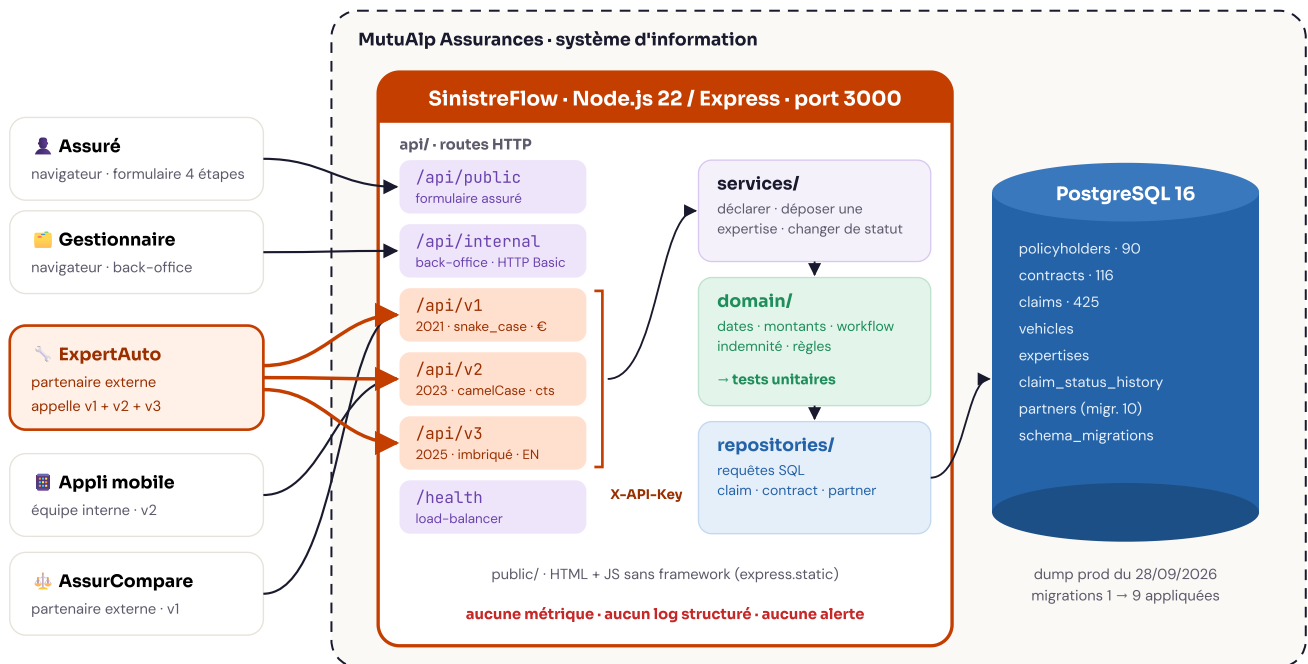
You build it, you run it

L'équipe qui écrit le code est celle qui le déploie et qui est réveillée quand il casse. On écrit donc du code facile à exploiter.

Comment mesure-t-on une équipe DevOps ? (métriques DORA)

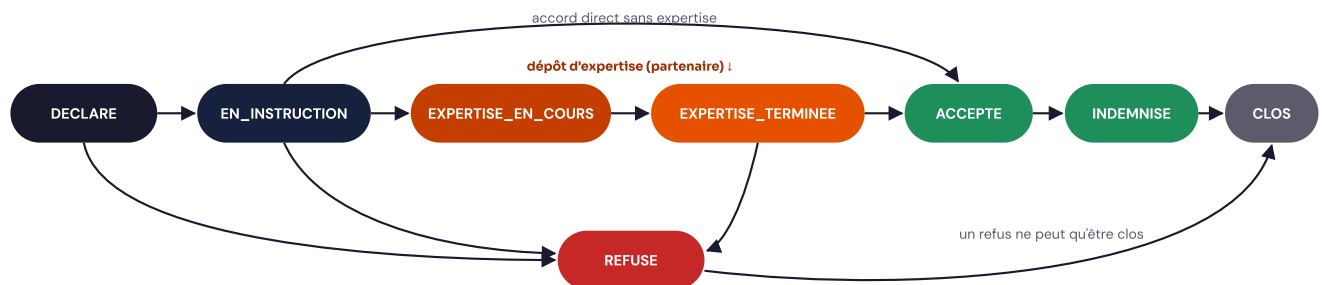
Métrique	Question	Chez MutuAlp aujourd'hui	Objectif vendredi
Fréquence de déploiement	À quelle fréquence livre-t-on en production ?	« quand Thomas avait le temps »	à chaque fusion sur main
Délai de mise en production	Combien de temps entre un commit et sa mise en prod ?	des semaines	< 30 min
Taux d'échec des changements	Quelle proportion de livraisons casse la prod ?	inconnu (aucune mesure)	mesuré, < 15 %
Temps de restauration	Combien de temps pour revenir à un état sain ?	inconnu (personne n'est prévenu)	< 10 min (alerte + retour arrière)

05 L'application SinistreFlow



Vue des composants. Une seule application sert le front, le back-office et trois versions d'API. Les flèches orange sont les appels du partenaire ExpertAuto : retenez qu'il utilise **les trois versions**. Seule la couche `domain/` est indépendante du web et de la base : c'est elle qu'on teste unitairement.

Le cycle de vie d'un dossier



Machine à états. Chaque flèche est une transition autorisée (`src/domain/workflow.js`). Règle du service Indemnisation : **seul un dossier accepté peut être indemnisé.**

Les données ont une histoire

La base de production a été construite par **10 migrations** successives sur 5 ans. Le dump fourni est à la version 9 ; la migration 10 n'a jamais été jouée en production. Lisez les fichiers de

migrations/ **dans l'ordre**, avec leurs commentaires : ils racontent des décisions qui expliquent plusieurs bugs.

06 Mission 0 - Prise en main

JOUR 1 - MATIN

Objectif : avoir l'application qui tourne sur votre poste avec les données de production, voir le connecteur partenaire échouer, et savoir reproduire chaque ticket.

- 1 Créer le dépôt de l'équipe.** Un membre crée un dépôt GitHub *privé* à partir du code fourni, invite les coéquipiers et le formateur. Dans *Settings* → *Branches*, protégez `main` : Pull Request obligatoire, 1 approbation, pas de push direct (les status checks seront ajoutés en mission 5).
- 2 Premier commit d'équipe.** Créez `docs/JOURNAL.md` (composition de l'équipe, rôles, objectifs du jour) via une PR.
- 3 Lancer la stack « comme Thomas ».**

```
npm install
docker compose up -d db
bash db/restore.sh           # restaure le dump de production
npm run migrate              # joue la migration 10
npm start                    # http://localhost:3000
```

Sous Windows, utilisez Git Bash ou WSL pour les scripts `.sh`.

- 4 Explorer.** Formulaire (`/`), back-office (`/backoffice.html`), `/health`. Comptes de démonstration dans `db/README.md`. Explorez la base : `docker compose exec db psql -U sinistreflow` puis `\dt`, `\d claims` ...
- 5 Lancer le connecteur partenaire** et constater l'échec. Copiez la sortie dans le journal : c'est votre « état zéro ».

```
export EXPERTAUTO_API_KEY=ea_live_7f3c9a1e5b2d4f60 # fournie par MutuAlp (voir docs)
python3 partner-client/expertauto_sync.py ; echo "code retour : $?"
```

- 6 Lancer les tests existants** (`npm test`). Ils passent... Est-ce une bonne nouvelle ?
- 7 Cartographier.** Dans `docs/CARTOGRAPHIE.md` : un schéma de qui appelle quoi, la liste des tables, et un tableau *ticket* → *reproduit ?* → *commande de reproduction* (`curl`, `écran`) → *fichier suspect*.



Reproduire avant de corriger. Un bug que vous ne savez pas reproduire, vous ne saurez pas prouver que vous l'avez corrigé. Gardez vos commandes `curl` : elles deviendront vos tests d'intégration.

```
curl -s -H "Content-Type: application/json" \
  -d '{"contractNumber":"MA-AUTO-001001","email":"camille.durand@example.test"}' \
  http://localhost:3000/api/public/contracts/verify
```

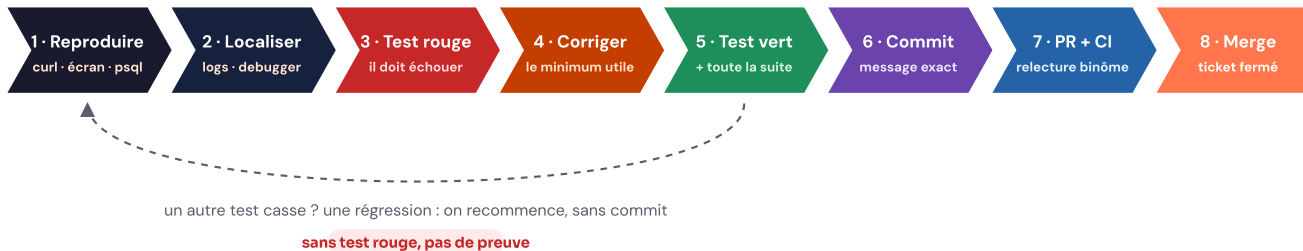
✓ Point de contrôle

- ☐ Dépôt GitHub créé, `main` protégée, formateur invité
- ☐ Application lancée localement avec le dump (425 dossiers visibles en base)
- ☐ Sortie du connecteur ExpertAuto (en échec) consignée dans le journal
- ☐ Cartographie et tableau de reproduction des tickets commencés

07 Mission 1 - Corriger les bugs

JOURS 1 → 2

Les tickets sont rédigés par des utilisateurs : ils décrivent ce qu'ils *voient*, jamais la cause. Votre travail est de remonter du symptôme à la cause, de la prouver par un test, puis de la corriger, proprement, un ticket à la fois.



La boucle de correction. Le test est écrit *avant* le correctif et doit échouer pour la bonne raison (le symptôme du ticket). C'est la seule preuve que votre correctif corrige bien ce bug, et la garantie qu'il ne reviendra pas.

Outils pour localiser

- Les logs de l'application (et leur absence !)
- `node --inspect src/index.js` + Chrome DevTools, ou le débogueur VS Code
- Les DevTools du navigateur, onglet *Network*
- `psql : \d claims`, `SELECT ...` sur les données réelles
- `git log -p fichier`, `git blame` : qui a changé quoi, et pourquoi
- Les commentaires des migrations SQL

Conseils d'organisation

- Répartissez les tickets dans un tableau *GitHub Projects* (À faire / En cours / En revue / Terminé).
- Certains bugs en masquent d'autres : SF-114 rend tous les codes d'erreur faux, SF-102 cache SF-103...
- Les tickets signalés  bloquent le partenaire : traitez-les en premier.
- Les étoiles indiquent la difficulté estimée (★ à ★★★★★).



Les indices sont repliés. Ouvrez-les seulement après avoir vraiment cherché. Notez dans le journal les indices ouverts : cela fait partie de l'honnêteté professionnelle.

Lot A - Application & API

SF-101 Le connecteur ExpertAuto est refusé

BLOQUANT

☆☆☆

Tous nos appels à vos API v1/v2/v3 échouent (erreur 500, message « Clé API manquante »). Nous envoyons pourtant bien l'en-tête `X-API-Key`. — DSI ExpertAuto

```
fix(SF-101): lire l'en-tête X-API-Key sans tenir compte de la casse
```

SF-102 « La date du sinistre ne peut pas être dans le futur »

BLOQUANT

☆☆☆

Impossible de déclarer un sinistre survenu hier ou avant-hier, le formulaire répond que la date est dans le futur. Les sinistres plus anciens passent. ExpertAuto signale le même message sur la date de rendez-vous d'expertise.

```
fix(SF-102): corriger le mois dans parseIsoDate
```

SF-103 Tous les dossiers sont « déclaration tardive »

MAJEUR

☆☆☆

Un dégât des eaux survenu la veille est signalé « déclaration tardive » alors que l'assuré a 5 jours pour déclarer (2 jours pour un vol). (Peut n'apparaître qu'une fois SF-102 corrigé.)

```
fix(SF-103): calculer le délai de déclaration en jours et non en heures
```

SF-104 Erreur à l'envoi de la déclaration

BLOQUANT

☆☆☆

Plus AUCUNE déclaration en ligne n'aboutit : « duplicate key value violates unique constraint ». En recette (base vide) ça marchait.

```
fix(SF-104): générer les références de dossier avec claim_reference_seq
```

SF-105 Un dossier refusé a été indemnisé

MAJEUR

☆☆☆

Le back-office propose le bouton « → INDEMNISE » sur les dossiers refusés. Le test automatique de l'ancienne équipe valide pourtant ce comportement ?! — Audit interne

```
fix(SF-105): interdire la transition REFUSE vers INDEMNISE
```

SF-106 Montants d'indemnité aberrants

BLOQUANT

☆☆☆

*Expertise à 1 000 €, franchise 150 € : SinistreFlow calcule 998,50 € au lieu de 850 €. Sur un bris de glace à 120 € avec 150 € de franchise, l'indemnité est **négative**.*

```
fix(SF-106): convertir la franchise en centimes et borner l'indemnité à zéro
```

SF-107 Listes vides ou incomplètes

MAJEUR

☆☆☆

Back-office, filtre EXPERTISE_EN_COURS : la liste est vide alors que le compteur annonce 14 dossiers. ExpertAuto ne reçoit plus aucun dossier à expertiser.

```
fix(SF-107): corriger le calcul de l'offset de pagination
```

SF-108 Recherche back-office : erreur 500 + alerte sécurité

SÉCURITÉ

☆☆☆

Rechercher l'assurée « D'Almeida » provoque syntax error at or near "Almeida". Le pentest a obtenu la liste complète des dossiers avec `%' OR 1=1 --` . — RSSI

```
fix(SF-108): paramétrer la requête de recherche du back-office
```

SF-109 Immatriculation absente dans l'API

BLOQUANT · ARCHITECTURE

★★★

Pour tous les dossiers récents, `immatriculation` (v1) et `vehiculePlate` (v2) valent `null`. En v3 la plaque est bien là. — CTO : « On supprime v1 et v2, tout le monde passe en v3, problème réglé. »

`fix(SF-109)`: remonter l'immatriculation depuis `vehicles` dans les API v1 et v2

→ Ce ticket fait l'objet de la [Mission 2](#) (analyse + ADR obligatoire).

SF-110 Montants renvoyés sous forme de texte

MAJEUR

★★★

En v2, `estimatedAmountCents` arrive sous la forme `"125050"` (chaîne) et non `125050` (nombre). Notre validation de schéma rejette la réponse. — ExpertAuto, équipe mobile

`fix(SF-110)`: renvoyer les montants BIGINT en nombres entiers

SF-111 Les dates reculent d'un jour

MAJEUR

★★★

Nous déposons un rapport avec un rendez-vous le 02/10, la relecture du dossier affiche le 01/10. L'écran de confirmation du formulaire affiche aussi la veille. (Le développeur qui a testé sur la CI dit qu'il ne reproduit pas.)

`fix(SF-111)`: supprimer le décalage de fuseau horaire sur les dates

SF-112 « Aucun contrat ne correspond »

MAJEUR

★★★

Mme Camille Durand (contrat MA-AUTO-001001) ne peut pas s'identifier avec `camille.durand@example.test`, l'adresse de son contrat. Environ un quart des assurés seraient concernés.

`fix(SF-112)`: comparer l'email de l'assuré sans tenir compte de la casse

SF-113 Montant estimé faux MINEUR

☆☆☆

L'assuré saisit `1 250,50` € et le dossier affiche 1,00 €. Pour `19.99` on obtient 19,98 €.

```
fix(SF-113): accepter les montants saisis au format français
```

SF-114 Erreurs 500 partout + fuite d'informations SÉCURITÉ

☆☆☆

Toutes les erreurs remontent en HTTP 500, même un dossier inexistant ou une saisie invalide. Les réponses contiennent la stack trace complète (chemins du serveur, versions des librairies). — RSSI

```
fix(SF-114): renvoyer le bon code HTTP et masquer la stack trace
```

SF-115 /health dit « UP » alors que tout est en panne MINEUR

☆☆☆

Lors de la panne PostgreSQL du 14/09, `/health` répondait `{"status": "UP"}` pendant que toutes les pages étaient en erreur. Le load-balancer n'a rien détecté. — Exploitation

```
fix(SF-115): vérifier la base de données dans le endpoint /health
```

Lot B - Front

SF-301 Impossible de déclarer un vol ou un cambriolage BLOQUANT

☆☆☆

Pour un cambriolage, le formulaire refuse l'envoi : « Le numéro de dépôt de plainte est obligatoire pour un vol », alors que l'assuré l'a bien saisi.

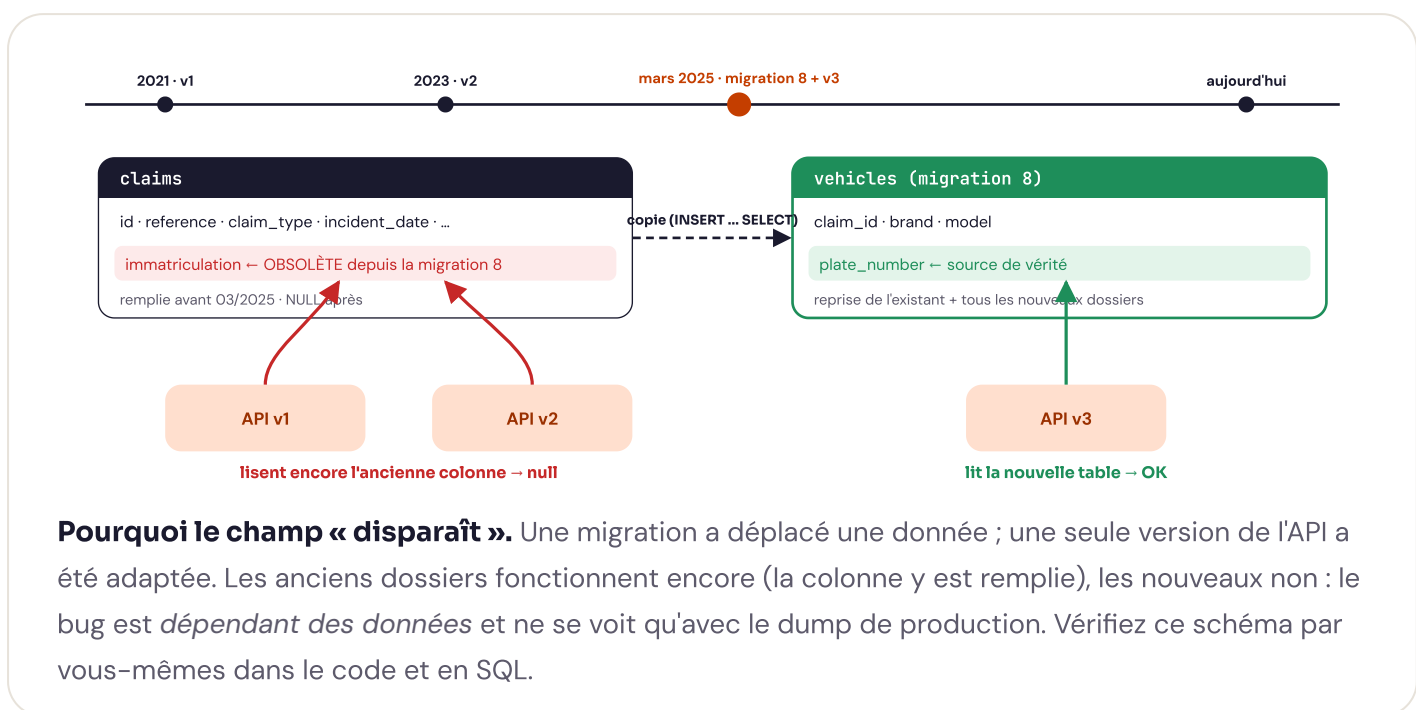
```
fix(SF-301): transmettre le numéro de plainte saisi dans le formulaire
```

08 Mission 2 - Le dilemme des versions (SF-109)

JOUR 2

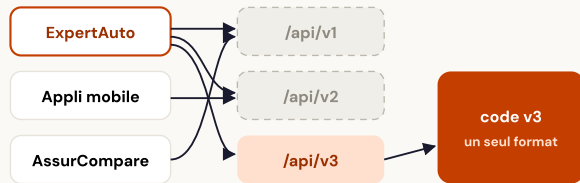
Un champ ne remonte plus dans deux versions de l'API. Le CTO veut profiter de l'occasion pour supprimer les anciennes versions. Avant de toucher au code, vous devez **comprendre, décider** et **documenter** la décision : c'est ce que fait une équipe DevOps responsable de sa production.

1. Comprendre la cause : les données ont bougé, pas tout le code



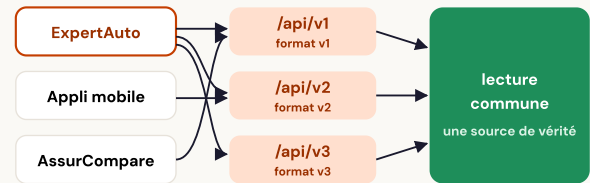
2. Deux options sur la table

Option A - « On ne garde que la v3 »



+ une seule version à maintenir
? que reçoivent les consommateurs de v1 et v2 ?
? et si l'on « redirige » v1 et v2 vers la v3 ?

Option B - « On garde toutes les versions »



+ aucun consommateur n'est impacté
? combien coûte la maintenance de 3 formats ?
? comment retirer un jour une version ?

À vous de trancher. Les deux options sont défendables sur le papier. Votre décision doit s'appuyer sur des **faits** : qui consomme quoi (`docs/API.md`), qui peut modifier son code, en combien de temps, ce que dit le contrat avec le partenaire... et ce que fera le connecteur ExpertAuto, qui ne sera pas modifié.

3. Notions à maîtriser

Changement cassant (breaking change)

Toute modification qui peut faire échouer un client existant *sans qu'il ait rien changé* : supprimer ou renommer un champ, changer son type (`"125050"` au lieu de `125050` ...) ou son unité, changer un code de statut, rendre un champ obligatoire, supprimer une route. Un changement cassant impose une **nouvelle version**.

Cycle de vie d'une version

Active → **Dépréciée** (on prévient : documentation, en-têtes HTTP `Deprecation` et `Sunset`, RFC 8594) → **Retirée**, quand on a *mesuré* qu'il n'y a plus de trafic. On ne retire jamais une version « parce qu'on pense » que personne ne l'utilise.

Tests de contrat

Pour chaque version : la liste exacte des champs, leurs types, leur format. Si un développeur modifie le schéma ou une requête, le test de contrat de la version concernée passe au rouge *avant* la production. C'est exactement ce qui a manqué en mars 2025.

Architecture Decision Record (ADR)

Un court document versionné avec le code : contexte, options, décision, conséquences. Dans deux ans, quelqu'un se demandera « pourquoi garde-t-on trois versions ? » : la réponse sera dans `docs/adr/`, pas dans la tête de quelqu'un qui est parti.

4. Ce qui est attendu

- 1 Analyse.** Retrouvez la cause exacte dans le code et prouvez-la en SQL (combien de dossiers concernés ? depuis quand ?).
- 2 Décision.** Rédigez `docs/adr/0001-versioning-api.md` avec le modèle ci-dessous (jalón du jour 2). Commit : `docs(SF-603): ADR 0001 stratégie de versioning de l'API`
- 3 Correctif.** Implémentez la décision. Commit : `fix(SF-109): remonter l'immatriculation depuis vehicles dans les API v1 et v2`
- 4 Garde-fous.** Tests d'intégration de contrat pour *chacune* des versions servies, et un test de cohérence : les versions servies décrivent le même dossier de la même façon (plaque, date, montants).
- 5 Bonus.** En-têtes de dépréciation sur la ou les versions à retirer à terme ; métrique d'usage par version et par partenaire (mission 7).

```
# docs/adr/0001-versioning-api.md
# ADR 0001 – Stratégie de versioning de l'API partenaires
- Statut : proposé | accepté | remplacé par ...
- Date, décideurs, ticket
```

```
## Contexte      ← les faits : consommateurs, contraintes contractuelles, cause d
## Options étudiées ← au moins 2, avec avantages / inconvénients / risques
## Décision      ← ce qu'on fait, en une phrase, puis le détail
## Conséquences   ← ce que ça coûte, ce que ça impose à l'équipe, le plan pour la
```

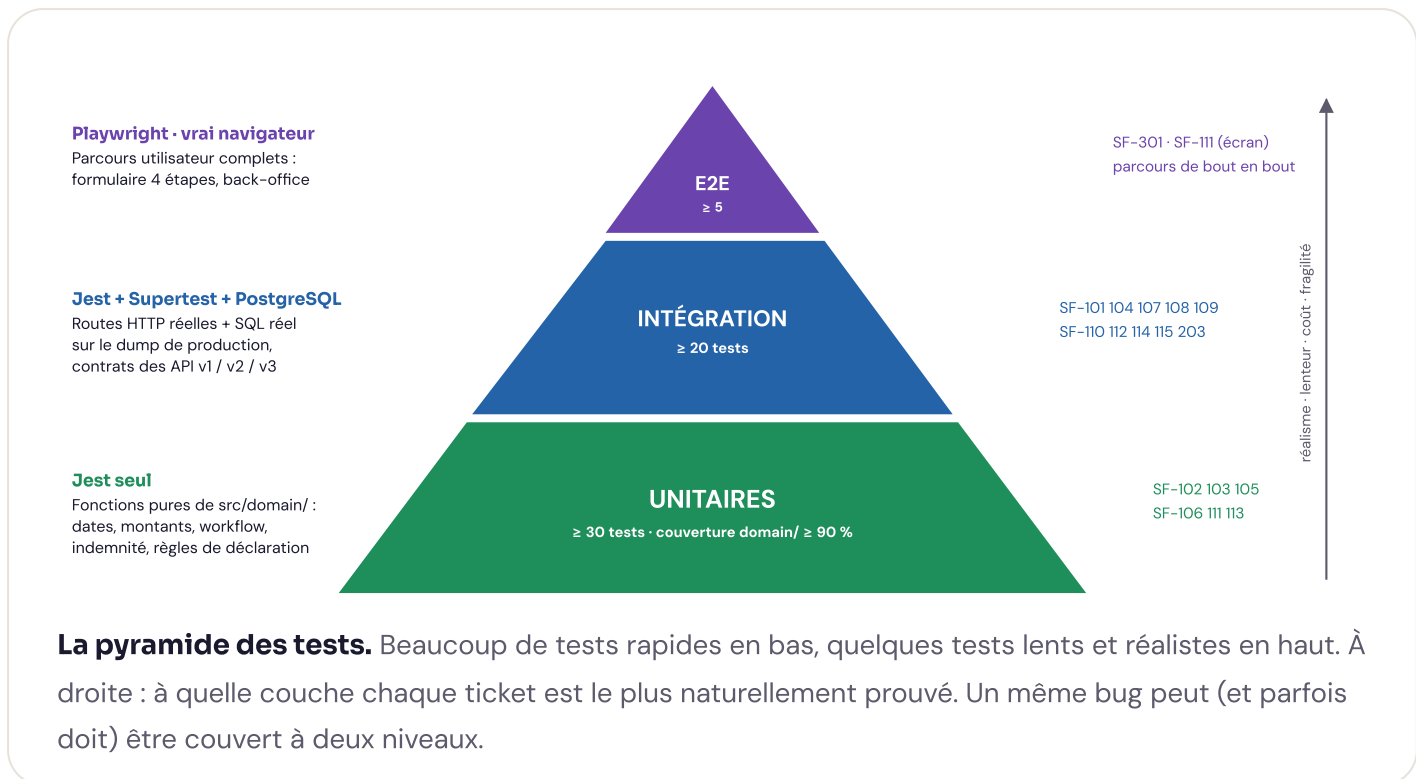


Rappel : le connecteur ExpertAuto ne sera pas modifié. Il doit passer au vert tel quel, dans la CI et sur votre VM. Une décision d'architecture se juge sur ses conséquences en production.

09 Mission 3 · Les 3 couches de tests

JOURS 1 → 3

Un test répond à une question : « est-ce que ça marche encore ? ». Plus on monte dans la pyramide, plus la réponse est réaliste... et plus elle est lente, chère et fragile. Une bonne stratégie combine les trois couches.



Couche 1 · Tests unitaires

Ils testent une fonction isolée, sans base, sans réseau, sans serveur. Exécution en millisecondes : on les lance à chaque sauvegarde.

```
// tests/unit/indemnity.test.js
const { computeIndemnityCents } = require('../src/domain/indemnity');

test('SF-106 : la franchise (en euros) est déduite en centimes', () => {
  expect(computeIndemnityCents(100000, 150)).toBe(85000); // 1 000 € - 150 € = 850
});

test.each([[ '1250', 125000 ], [ '19.99', 1999 ] ])( '%s → %i centimes', (saisie, attendu)
```

☐ ≥ 30 tests sur `src/domain/`, couverture des lignes ≥ 90 % (`jest --coverage`)

☐ Les cas limites : J+5 / J+6, 29 février, montant nul, franchise > montant, saisie invalide...

- ☐ Fuseau horaire fixé dans les tests qui manipulent des dates (`process.env.TZ = 'Europe/Paris'`)

- ☐ Le test historique faux (SF-105) corrigé

Couche 2 - Tests d'intégration

Ils démarrent l'application Express *en mémoire* (Supertest) et lui envoient de vraies requêtes HTTP, qui exécutent du vrai SQL sur une **vraie base PostgreSQL restaurée depuis le dump**. C'est la couche qui détecte les bugs « ça marche sur base vide ».

```
// tests/integration/api-v2.test.js
const request = require('supertest');
const app = require('../src/app'); // pas de app.listen : Supertest s'en
const db = require('../src/db/pool');
afterAll(() => db.pool.end());

test('SF-110 : contrat v2, les montants sont des entiers JSON', async () => {
  const res = await request(app).get('/api/v2/claims/SIN-2026-000450').set('X-API-Key');
  expect(res.status).toBe(200);
  expect(Number.isInteger(res.body.estimatedAmountCents)).toBe(true);
});
```

- ☐ ≥ 20 tests, base restaurée depuis le dump puis migrée avant la suite
- ☐ Un test de **contrat** par version d'API servie (liste exacte des champs + types)
- ☐ Codes HTTP : 400, 401, 404, 409 — et jamais de stack trace dans la réponse
- ☐ Toutes les migrations appliquées sur une base **vide** (SF-203)
- ☐ Script `npm run test:integration` (séparé des unitaires : `jest --selectProjects` ou deux configs)

Couche 3 - Tests End-to-End

Un vrai navigateur (Chromium piloté par [Playwright](#)) remplit le formulaire comme un assuré. C'est la seule couche qui voit un bug dans le JavaScript du front (SF-301) ou un affichage faux.

```
npm install -D @playwright/test && npx playwright install chromium
```

```
// tests/e2e/declaration.spec.js
test('un assuré déclare un accident auto', async ({ page }) => {
```

```
await page.goto('/');
await page.getByLabel('Numéro de contrat').fill('MA-AUTO-001001');
await page.getByLabel('Adresse email').fill('camille.durand@example.test');
await page.getByTestId('next-1').click();
// ... étapes 2, 3, 4 ...
await expect(page.getByTestId('reference')).toHaveText(/^SIN-\d{4}-\d{6}$/);
});
```

- ☐ ≥ 5 scénarios : accident auto, cambriolage avec plainte, contrat résilié, erreur de validation, back-office (recherche + changement de statut)

- ☐ `webServer` dans `playwright.config.js` pour démarrer l'application, fuseau `Europe/Paris`, locale `fr-FR`

- ☐ Traces / captures conservées en cas d'échec (artefact de CI)



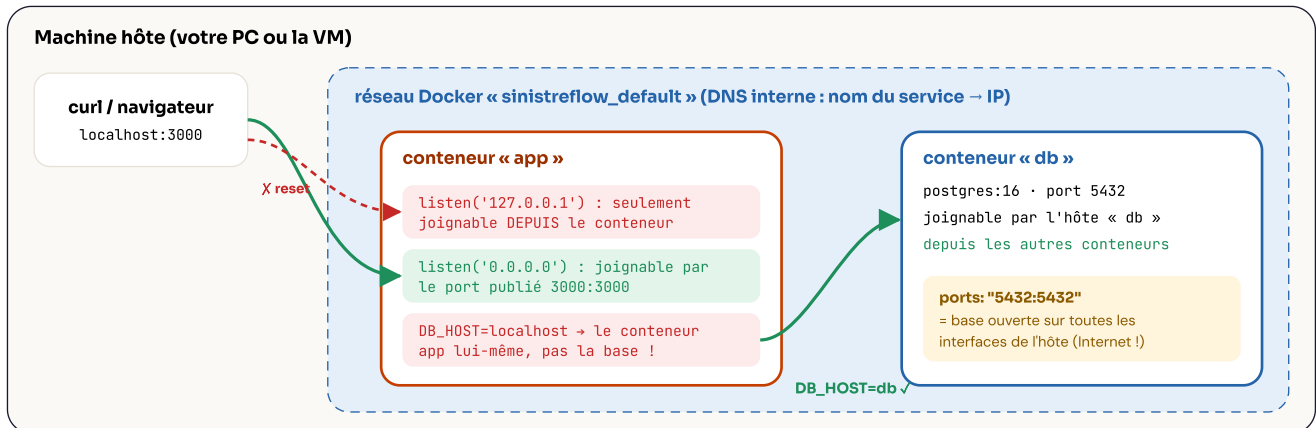
Commits attendus (en plus des tests livrés avec chaque correctif) : `test(SF-401):`

`tests unitaires du domaine métier` · `test(SF-402): tests d'intégration API`
`et base de données` · `test(SF-403): tests end-to-end Playwright`

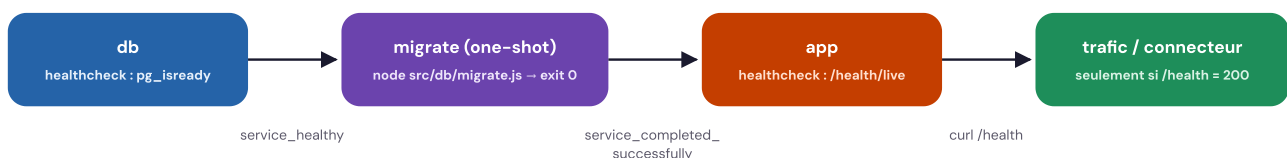
10 Mission 4 - Conteneurs & configuration (Lot C)

JOUR 3 - MATIN

Thomas lançait l'application avec `npm start` et utilisait Docker « juste pour la base ». Sur la VM, tout tournera en conteneurs. Objectif : `docker compose up -d --build` sur une machine neuve doit suffire.



Le piège de `localhost`. Dans un conteneur, `localhost` désigne le conteneur lui-même. Un serveur qui n'écoute que sur `127.0.0.1` est invisible de l'extérieur, et un service qui cherche la base sur `localhost` ne la trouve pas. Entre conteneurs d'un même projet compose, on utilise le **nom du service**.



depends_on: [db] seul n'attend que le DÉMARRAGE du conteneur, pas que PostgreSQL soit prêt

Ordonner le démarrage. Chaque étape attend que la précédente soit réellement prête. Les migrations deviennent une étape du déploiement, plus une commande qu'il faut « penser à lancer » (SF-206).

SF-201 Application injoignable dans Docker

BLOQUANT

★★☆

Conteneur `app` démarré, logs OK (« SinistreFlow démarré sur `http://localhost:3000` »), mais `curl http://localhost:3000` depuis l'hôte : `connection reset / empty reply`.

fix(SF-201): écouter sur toutes les interfaces réseau

SF-202 `docker compose up` : l'application ne trouve pas la base

BLOQUANT

★★☆

`connect ECONNREFUSED 127.0.0.1:5432` dans le conteneur `app`. Au premier démarrage, l'application démarre avant que PostgreSQL soit prêt. L'audit réseau signale que PostgreSQL est joignable depuis l'extérieur.

fix(SF-202): configurer l'hôte de la base et le healthcheck dans docker compose

SF-203 Migrations impossibles sur une base neuve

MAJEUR

★★☆

`npm run migrate` sur une base vide : « `10_partners_api_keys.sql` : relation "expertises" does not exist ». Sur la base de prod ça passe.

fix(SF-203): trier les migrations par numéro de version

SF-204 Secrets dans le dépôt Git

SÉCURITÉ

★★☆

Le mot de passe de la base de production, celui du back-office et la clé API d'ExpertAuto sont lisibles dans le dépôt (et dans tout l'historique Git). Des valeurs par défaut sont aussi codées en dur. — RSSI

fix(SF-204): retirer les secrets du dépôt

SF-205 Image Docker de plus d'1 Go, exécutée en root

MINEUR

☆☆☆

L'image contient les dépendances de dev, le fichier `.env`, les tests, le dump de la base... et le process tourne en root.

`fix(SF-205): image Docker de production légère et sans root`

SF-206 Erreur 500 après chaque mise en production

BLOQUANT

☆☆☆

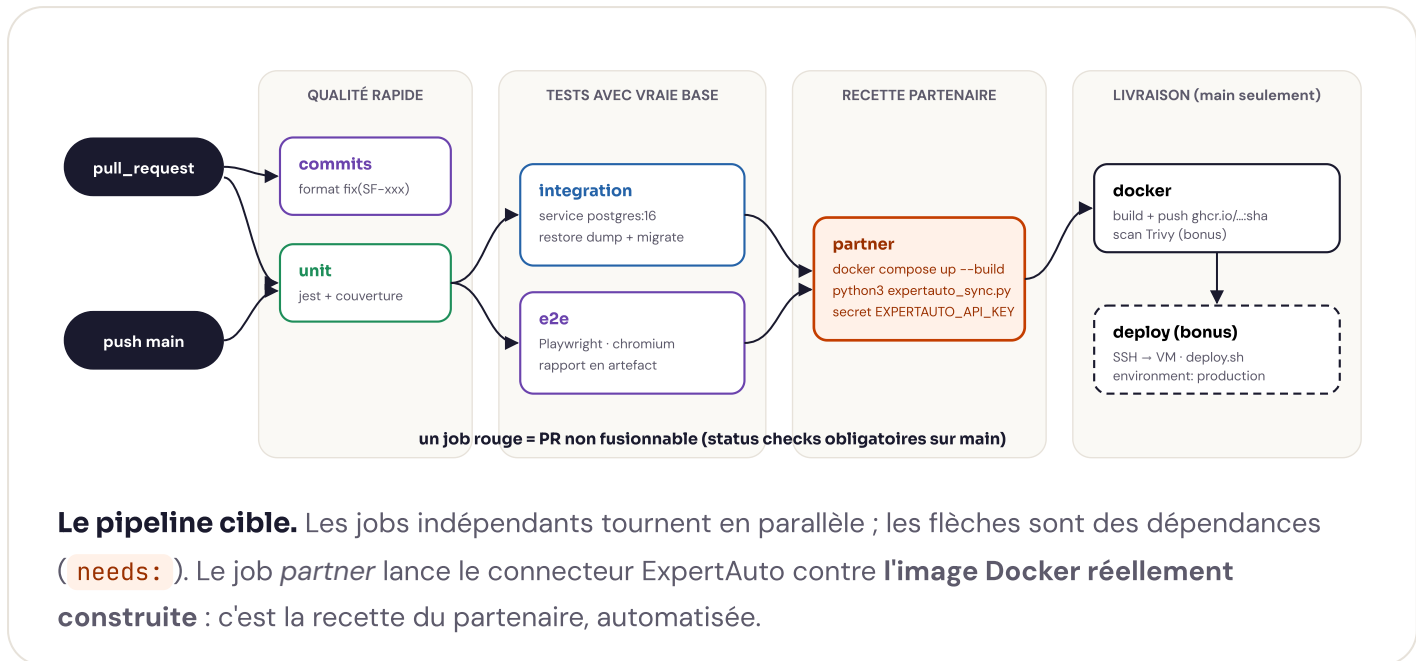
Après la dernière livraison : « relation "partners" does not exist ». Personne n'a lancé les migrations. Ça ne doit plus dépendre de la mémoire de quelqu'un.

`fix(SF-206): jouer les migrations à chaque déploiement`

11 Mission 5 · Intégration continue avec GitHub Actions

JOUR 3 · APRÈS-MIDI

L'intégration continue (CI), c'est un robot qui, à chaque Pull Request, refait *exactement* ce que vous devriez faire à la main avant de fusionner : installer, tester à tous les niveaux, construire l'image, et vérifier que le partenaire fonctionne. Si le robot dit non, on ne fusionne pas.



Exigences

- ☐ Fichier `.github/workflows/ci.yml` déclenché sur `pull_request` et `push` sur `main`
- ☐ Jobs **unit**, **integration** (conteneur de service `postgres:16`, restauration du dump, migrations), **e2e** (Playwright, rapport en artefact si échec)
- ☐ Job **partner** : construit l'image, démarre la stack compose, exécute `partner-client/expertauto_sync.py` ; la clé est un secret GitHub
- ☐ Job **docker** (sur `main` uniquement) : image poussée sur GitHub Container Registry, taguée avec le SHA du commit
- ☐ Fuseau `TZ: Europe/Paris` dans les jobs de test, cache npm, versions d'actions figées (`@v4`)
- ☐ Protection de `main` : les jobs unit, integration, e2e et partner sont des *required status checks*

☐ Badge de statut du pipeline en haut du `README.md`

☐ Bonus : job de vérification des messages de commit, scan de vulnérabilités, job *deploy* avec environnement protégé

```
# .github/workflows/ci.yml – point de départ
name: CI SinistreFlow
on:
  pull_request:
  push:
    branches: [main]
env:
  TZ: Europe/Paris
jobs:
  unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run test:unit
# integration: à vous (indice : jobs.<id>.services.postgres + psql -f db/dump/...)
# e2e, partner, docker : à vous
```



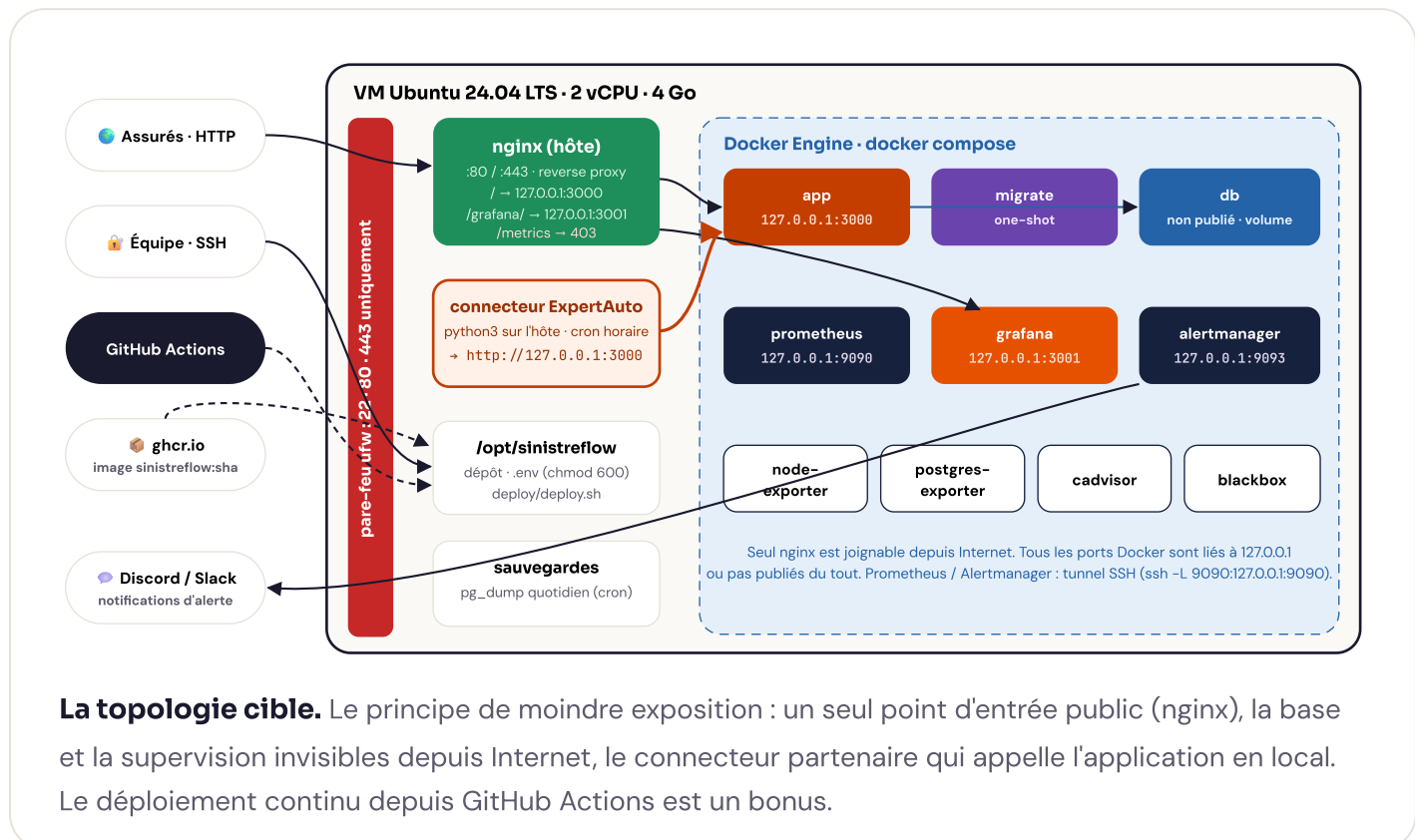
Commits attendus : `ci(SF-404): pipeline d'intégration continue GitHub`

Actions · `ci(SF-405): publier l'image Docker sur GHCR` . Pour mettre au point le pipeline sans polluer `main` , travaillez sur une branche et une PR « brouillon », puis regroupez proprement vos commits avant la fusion.

12 Mission 6 · Déploiement sur une VM

JOUR 4 · MATIN

La direction veut savoir si SinistreFlow peut être remonté sur un serveur neuf. Vous partez d'une VM Ubuntu vierge, vous la sécurisez, vous y déployez la stack, vous restaurez la base de production... et vous lancez le connecteur ExpertAuto **depuis la VM**. S'il est vert, la production est rétablie.



La topologie cible. Le principe de moindre exposition : un seul point d'entrée public (nginx), la base et la supervision invisibles depuis Internet, le connecteur partenaire qui appelle l'application en local. Le déploiement continu depuis GitHub Actions est un bonus.

Étapes

- 1 Obtenir la VM.** Fournie par le formateur, ou créée par vos soins (cloud, VirtualBox, Multipass) : Ubuntu 24.04 LTS, 2 vCPU, 4 Go de RAM, 20 Go de disque.
- 2 Sécuriser l'accès.** Connexion par clé SSH uniquement, utilisateur dédié `deploy` (groupe `docker`), `PermitRootLogin no`, `PasswordAuthentication no`, fail2ban, mises à jour de sécurité automatiques.
- 3 Pare-feu.** `ufw default deny incoming`, puis n'ouvrir que 22, 80, 443. Vérifiez depuis votre poste avec `nc -zv <ip> 5432` que la base est fermée.
- 4 Installer Docker** Engine et le plugin compose depuis le dépôt officiel Docker.

- 5 **Configurer.** Code dans `/opt/sinistreflow` (clé de déploiement GitHub en lecture seule, ou image GHCR), `.env` de production avec des secrets **neufs** (`openssl rand -base64 24`), `chmod 600 .env`.
- 6 **Première mise en service.** Démarrer la base, restaurer le dump, démarrer la stack (les migrations jouent toutes seules), vérifier `/health`.
- 7 **Reverse proxy** nginx devant l'application, `/metrics` bloqué. Bonus : HTTPS avec certbot si vous disposez d'un nom de domaine.
- 8 **Le moment de vérité.** Depuis la VM :

```
cd /opt/sinistreflow
SINISTREFLOW_URL=http://127.0.0.1:3000 EXPERTAUTO_API_KEY=... python3 partner-cli
# SYNCHRONISATION CONFORME : 25/25 contrôles OK ← capture d'écran dans le jou
```

- 9 **Automatiser.** Scripts versionnés : `deploy/provision-vm.sh` (étapes 2 à 4), `deploy/deploy.sh` (pull de l'image, migrations, démarrage, attente de `/health`, connecteur, **retour arrière automatique** sur l'image précédente si échec), et `docs/RUNBOOK.md` (comment déployer, restaurer, réagir à chaque alerte).



Retour arrière et migrations. Revenir à l'image précédente ne « dé-migre » pas la base. Une migration doit donc rester compatible avec la version N-1 du code : on ajoute une colonne, on ne la renomme ni ne la supprime dans la même livraison (stratégie *expand / contract*). C'est d'ailleurs ce qu'avait fait la migration 8... à moitié.

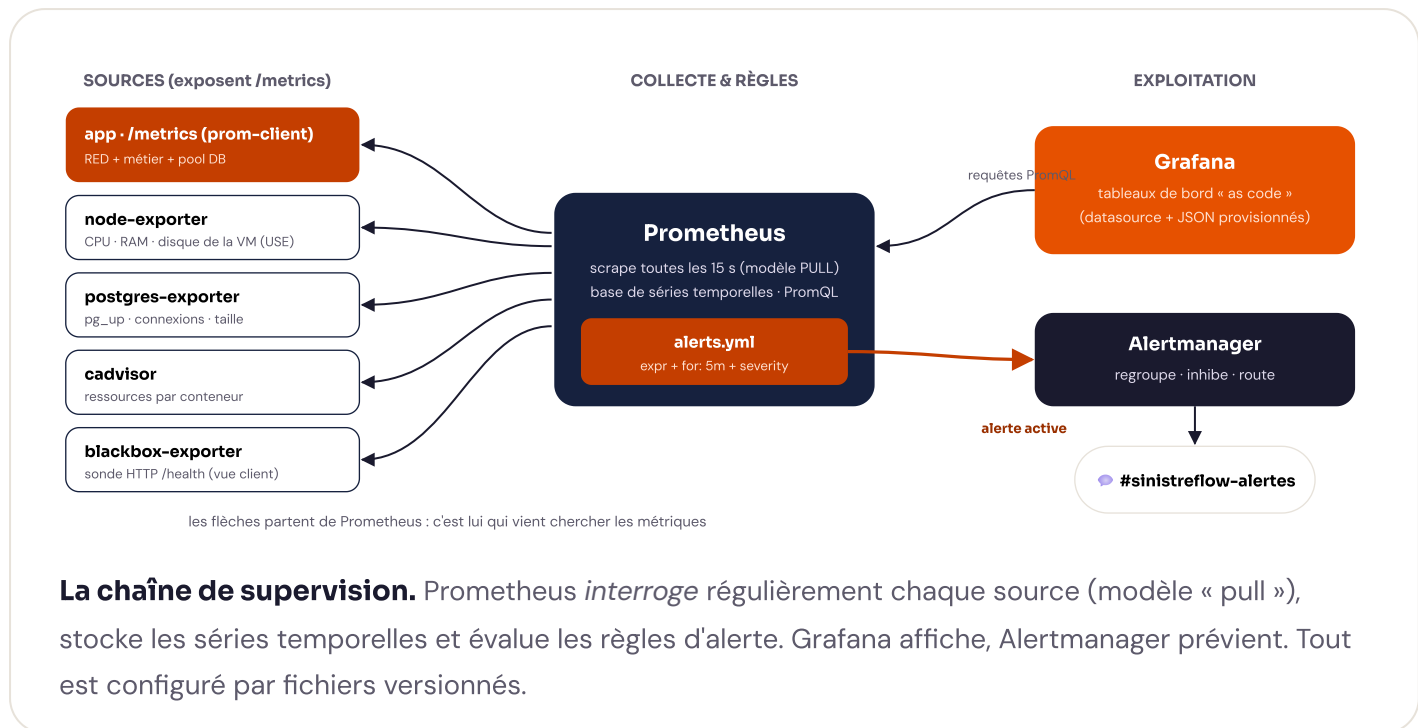


Le connecteur consomme des données. Chaque exécution réussie fait passer un dossier en `EXPERTISE_TERMINEE` (10 dossiers auto disponibles dans le dump). Pour rejouer à volonté sur la VM : `bash db/restore.sh` puis `docker compose run --rm migrate`.



Commits attendus : `feat(SF-601): script de provisionnement de la VM` · `feat(SF-602): script de déploiement avec retour arrière` · `docs(SF-604):`

Le 14 septembre, la base est tombée et personne ne l'a su pendant 3 heures. Objectif : que l'équipe soit **prévenue avant les utilisateurs**, et qu'elle puisse comprendre ce qui se passe en un coup d'œil.



Que mesurer ?

Les 4 signaux d'or

Latence (combien de temps), **trafic** (combien de requêtes), **erreurs** (combien échouent), **saturation** (à quel point on est plein). Issus du livre *Site Reliability Engineering* de Google.

RED pour les services

Rate, Errors, Duration : pour chaque route de l'API. Un compteur de requêtes étiqueté par code HTTP et un histogramme de durées suffisent à tout calculer.

USE pour les ressources

Utilization, Saturation, Errors : CPU, mémoire, disque, connexions à la base. Fourni par les exporters, sans code.

Type	Comportement	Exemple SinistreFlow	Requête Prometheus
Counter	ne fait que monter	<code>sinistreflow_http_requests_total{route,status}</code>	<code>sum by (route) (sinistreflow_http_requests_total[5m]))</code>
Gauge	monte et descend	<code>sinistreflow_db_pool_connections{state}</code>	<code>max_over_time(sinistreflow_db_pool_connections[5m])</code>
Histogram	répartition par tranches	<code>sinistreflow_http_request_duration_seconds</code>	<code>histogram_quantile(0.95, sum by (route) (sinistreflow_http_request_duration_seconds_bucket[5m]))</code>



Attention à la cardinalité. Une étiquette `route="/api/v2/claims/SIN-2026-000450"` crée une série par dossier : Prometheus explose. On étiquette avec le *motif* de la route (`/api/v2/claims/:reference`), jamais avec un identifiant.

Exigences

Livrable	Contenu minimal	Commit
Métriques applicatives	<code>GET /metrics</code> (prom-client) : compteur et histogramme HTTP par méthode / route / statut, métriques par défaut Node.js, état du pool PostgreSQL, et au moins 2 métriques métier (déclarations par type, expertises déposées, appels par version d'API et partenaire...)	<code>feat(SF-501):</code> <code>exposer les</code> <code>métriques</code> <code>Prometheus sur</code> <code>/metrics</code>
Stack de supervision	Fichier compose dédié : Prometheus, Grafana (datasource + tableau de bord provisionnés par fichiers), node-exporter, postgres-exporter. Rien d'exposé publiquement sauf Grafana derrière nginx (ou tunnel SSH).	<code>feat(SF-502):</code> <code>ajouter la stack</code> <code>Prometheus et</code> <code>Grafana</code>
Tableau de bord	État app / base / sonde, req/s, % d'erreurs 5xx, latence p95, déclarations et expertises (24 h), usage des versions d'API, CPU / RAM / disque de la VM	(dans SF-502)
Alertes	≥ 5 règles avec <code>for:</code> et <code>severity</code> : application injoignable, <code>/health</code> en échec ou base injoignable, taux de 5xx > 5 %, latence p95 > 1 s, disque < 10 %. Alertmanager → Discord, Slack ou e-mail.	<code>feat(SF-503):</code> <code>ajouter les règles</code> <code>d'alerte</code>
Journaux	Logs JSON une ligne par événement (horodatage, niveau, route, statut, durée, partenaire), erreurs 5xx avec stack <i>dans les logs</i> (pas dans la réponse). Bonus : Loki.	<code>feat(SF-504):</code> <code>produire des</code> <code>journaux structurés</code> <code>JSON</code>

Le test de panne (« game day »)

Une supervision qui n'a jamais sonné est une supervision dont on ne sait pas si elle marche. Sur la VM :

- 1 Panne de base.** `docker compose stop db` . Chronométrez : combien de temps avant que `/health` passe en 503, que l'alerte passe en *firing*, que la notification arrive ? Redémarrez, mesurez le temps de retour à la normale.
- 2 Pluie d'erreurs.** Une boucle de 200 appels avec une mauvaise clé API : quelle alerte, quel panneau le montre ?

- 3 **Mauvaise livraison.** Déployez volontairement une image cassée (variable obligatoire manquante, par exemple) : `deploy.sh` doit revenir seul à la version précédente.
- 4 **Post-mortem.** Pour chaque scénario, dans `docs/JOURNAL.md` : chronologie, captures Grafana / notification, temps de détection, temps de résolution, ce qu'il faudrait améliorer. Captures dans `docs/preuves/`.

Le rendu, c'est **votre dépôt GitHub, et rien d'autre**. Envoyez son lien au formateur (dépôt privé, formateur invité). La note est établie uniquement à partir de ce que contient le dépôt : ce qui n'y est pas n'existe pas.

L'historique Git

Les commits au message exact, les branches, les Pull Requests relues, la répartition du travail entre les membres. L'historique est vérifié automatiquement.

Le pipeline

L'onglet *Actions* doit montrer un pipeline vert sur `main`, recette du connecteur ExpertAuto comprise. Le formateur peut relancer le pipeline à tout moment.

Les preuves

Ce qui ne se voit pas dans le code se prouve dans `docs/preuves/` : sortie du connecteur au vert **sur la VM**, tableau de bord Grafana, alerte reçue, captures du game day, sortie de `ufw status`.

Check-list de fin de TP

- ☐ 22 commits de correction au message exact, chacun avec son test
- ☐ ADR `docs/adr/0001-versioning-api.md`
- ☐ Tests unitaires, d'intégration et E2E, lancés par la CI
- ☐ Pipeline GitHub Actions vert sur `main`, `main` protégée par les status checks
- ☐ Image publiée sur GHCR
- ☐ VM sécurisée, stack déployée, base restaurée, connecteur ExpertAuto vert **depuis la VM**
- ☐ Grafana avec tableau de bord, alertes testées, notification reçue

☐ `docs/RUNBOOK.md` , `docs/JOURNAL.md` , `docs/CARTOGRAPHIE.md`

☐ Captures et sorties dans `docs/preuves/`

☐ Aucun secret dans le dépôt (`git grep -i password` ne renvoie rien de sensible)

☐ Lien du dépôt envoyé au formateur

15 Barème

Domaine	Critères	Points
Corrections (lots A, B, C)	22 tickets : correctif juste, commit au message exact, test qui échouait avant et passe après	30
Versioning (SF-109)	ADR argumentée sur des faits, correctif cohérent avec la décision, tests de contrat par version	10
Tests	Unitaires (5) · intégration sur la vraie base (5) · End-to-End Playwright (5)	15
Intégration continue	Pipeline complet, recette partenaire automatisée, image GHCR, protection de branche	10
Conteneurs	Image légère et sans root, compose ordonné (healthchecks, migrations), aucun port superflu	5
Déploiement	VM sécurisée, scripts versionnés, retour arrière, connecteur vert depuis la VM, runbook	10
Supervision	Métriques applicatives et métier, tableau de bord, alertes, game day documenté	10
Pratiques Git & équipe	Branches, PR relues, journal de bord, répartition du travail visible dans l'historique	5
Documentation & preuves	README à jour (badge CI, lancement), runbook, journal, captures dans docs/preuves/	5
Total — note établie uniquement à partir du dépôt GitHub rendu		100

16 Annexes

Commandes utiles

```
# --- Git
git switch -c fix/SF-107-pagination
git add -p                # ajouter morceau par morceau
git commit -m "fix(SF-107): corriger le calcul de l'offset de pagination"
git log --oneline --grep "SF-"
git bisect start / good / bad    # trouver le commit fautif

# --- PostgreSQL
docker compose exec db psql -U sinistreflow
\dt    \d claims    \x
SELECT status, count(*) FROM claims GROUP BY 1;
SELECT * FROM schema_migrations;
```

```
# --- Docker
docker compose up -d --build
docker compose ps
docker compose logs -f app
docker compose exec app sh
docker compose run --rm migrate
docker images | grep sinistreflow
docker system df

# --- HTTP
curl -s -H "X-API-Key: $KEY" localhost:3000/api/v1/claims?limit=2 | jq
curl -si localhost:3000/health
```

Glossaire

Terme	Définition
CI / CD	Intégration continue (tester automatiquement chaque changement) / livraison ou déploiement continu (le mettre en production automatiquement).
ADR	Architecture Decision Record : document court qui trace une décision d'architecture, son contexte et ses conséquences.
Migration	Script versionné qui fait évoluer le schéma de la base. Toujours rejoué dans l'ordre, une seule fois.
Changement cassant	Modification d'une interface publique qui peut faire échouer un client existant.
Healthcheck · liveness · readiness	Sonde de santé. <i>Liveness</i> : le process vit-il ? <i>Readiness</i> : peut-il servir des requêtes (base joignable) ?
Reverse proxy	Serveur frontal (nginx) qui reçoit le trafic public et le relaie vers les applications internes.
Exporter	Petit programme qui expose au format Prometheus les métriques d'un système qui ne le fait pas lui-même.
SLO	Objectif de niveau de service, ex. « 99,5 % des déclarations aboutissent en moins d'une seconde sur 30 jours ».
Post-mortem	Analyse sans recherche de coupable d'un incident : chronologie, causes, actions d'amélioration.

Ressources officielles

- [GitHub Actions](#) · [Docker Compose](#) · [Bonnes pratiques Dockerfile](#)
- [Jest](#) · [Supertest](#) · [Playwright](#)
- [node-postgres](#) · [PostgreSQL 16](#)
- [Prometheus](#) · [Grafana](#) · [prom-client](#)
- [Google SRE : Monitoring Distributed Systems](#) · [DORA](#) · [ADR](#)



IT-Akademie · Module DevOps · TP « SinistreFlow » · MutuAlp Assurances est une entreprise fictive,
toutes les données sont générées.

Apprenez la tech. Construisez votre avenir.